```
template<typename Iterator,typename T>
struct accumulate_block
{
    void operator()(Iterator first,Iterator last,T& result)
    {
        result=std::accumulate(first,last,result);      <-1
    }
};

template<typename Iterator,typename T>
T parallel_accumulate(Iterator first,Iterator last,T init)
{
    unsigned long const length=std::distance(first,last);      <-2

    if(!length)
        return init;

    unsigned long const min_per_thread=25;
    unsigned long const max_threads=
        (length+min_per_thread-1)/min_per_thread;


    unsigned long const hardware_threads=
        std::thread::hardware_concurrency();

    unsigned long const num_threads=
        std::min(hardware_threads!=0?hardware_threads:2,max_threads);

    unsigned long const block_size=length/num_threads;

    std::vector<T> results(num_threads);            <-3
    std::vector<std::thread>  threads(num_threads-1);      <-4

    Iterator block_start=first;                     <-5
    for(unsigned long i=0;i<(num_threads-1);++i)
    {
        Iterator block_end=block_start;      <-6
        std::advance(block_end,block_size);
        threads[i]=std::thread(              <-7
            accumulate_block<Iterator,T>(),
            block_start,block_end,std::ref(results[i]));
        block_start=block_end;                                  <-8
    }
    accumulate_block()(block_start,last,results[num_threads-1]);     <-9

    std::for_each(threads.begin(),threads.end(),
        std::mem_fn(&std::thread::join));

    return std::accumulate(results.begin(),results.end(),init);     <-10
}
```

Now let’s go through and identify the places where an exception can be thrown: basically anywhere where you call a function you know can throw